

FAYE Cheikh Bamba
GULLIENT Anthony



Compte-rendu jeux vidéo

Pour la réalisation d'un jeu 3D, il existe différents outils. Pour notre jeu, on a utilisé Irrlicht qui est un moteur 3D. Il s'agit d'une bibliothèque logicielle qui va s'occuper de tout ce qui concerne l'affichage à l'écran, les calculs dans l'espace, la lecture des fichiers externes.

Nous allons ainsi dans un premier temps vous présenter le jeu puis parler de la conception du jeu à savoir, les fonctionnalités, la création des fichiers, des classes et autres. Puis, on va parler des perspectives pour le jeu.

I. Présentation du jeu

Notre jeu "SURVIVE" est un jeu d'action. Le joueur devra survivre aux nombres vagues des zombies et les tuer tous durant un temps imparti. Au fil des différents vagues de zombies, la difficulté du jeu sera augmentée et vous découvrirez des zombies du plus en plus coriaces. Notre jeu propose une aventure dans la map suivante :



Pour lancer le jeu, il faudra lancer l'exécutable `./survive` depuis un terminal et le jeu pourra démarrer. Les règles du jeu et les commandes sont disponibles sur le jeu via des boutons qu'on a créé. En résumé, les instructions sont d'éliminer toutes les vagues de zombies avant que le temps imparti finisse. A l'aide d'une arme (AK-47) avec une infinité de cartouches, vous pourriez éliminer les zombies. Les zombies seront sensibles votre odeur, suivant la distance qui vous sépare, ils peuvent vous détecter et vous attaquer.

II. Conception du jeu

Pour avoir un structure du projet bien organisé, on a décidé de créer différentes classes, une classe *Game* pour la gestion du jeu, une classe *camera*, une classe *player* pour la gestion des armes, une classe *Enemy* pour la création des zombies et une classe *GameEvent* pour la gestion des interactions entre le clavier, la souris et le jeu. Nous allons détailler leur fonction en expliquant quelques méthodes et fonctions de ces classes.

1. Camera:

Nous avons créé une classe *Camera*, qui a comme attribut principale une variable de type *ICameraSceneNode* représentant un node qui permet de stocker une caméra. Il s'agit d'une caméra FPS dont l'orientation est gérée par la souris et les déplacements par le clavier. On pourra alors raccorder directement l'arme à la caméra pour simplifier les calculs. Pendant la création du node, on peut assigner un paramètre *keymap* qui est une "carte" permettant d'assigner des touches à certaines actions. Cela nous a alors permis de changer les touches de déplacements.

On a créé des setteurs et des getteurs pour la classe pour avoir accès à ce node et en plus de cela, on a implémenté l'effet de la gravité et la collision entre la caméra et le map en utilisant un selector qui va sélectionner le maillage du map puis à l'aide d'une animation de type *createCollisionResponseAnimator()*, on empêche le node de traverser les murs.

2. Enemy:

La classe *Enemy*, contient tous les éléments liés aux ennemis. Dans un premier temps, le constructeur nous permet de charger le mesh animé qui va nous servir de base pour représenter l'ennemi. Nous lui donnons une position, une taille et nous réglons la fréquence d'animation. Dans cette fonction, nous initialisons aussi tous les paramètres pour un bon affichage, nous importons et appliquons la texture puis nous faisons en sorte que la lumière soit correcte. De plus, nous lui appliquons des points de vie, une vitesse et une cible (le joueur).

Cette classe nous permet aussi de gérer le comportement des ennemis, comme leur déplacement dans la fonction *attack*. Cette fonction assigne une nouvelle position et rotation à l'ennemi en fonction du positionnement du joueur. Elle fait appel à des getteurs de la même classe qui permettent de connaître la distance avec le joueur ainsi que la direction et l'orientation à adopter.

Nous avons aussi la méthode *collision* permettant, comme son nom l'indique, de gérer les collision en utilisant les bounding box des nodes et les fonctions *createCollisionResponseAnimator* et *setTriangleSelector*.

De plus, nous avons implémenté des fonctions permettant de modifier le node associé à l'ennemi lorsque celui-ci perd de la vie, *healthDecrease* et *enemyHurt*. Elles nous permettent de jouer une nouvelle animation ainsi que changer sa texture lorsque l'ennemi est blessé.

Pour finir, nous avons un getteur permettant l'accès au nombre de points de vie de l'ennemi.

3. Player:

Dans cette classe, nous initialisons le joueur ainsi que toutes les méthodes permettant d'interagir avec ce dernier.

Nous avons choisi un jeu en format FPS pour un ressenti plus immersif. Pour cela, nous avons utilisé une caméra FPS fournie par Irrlicht à laquelle nous avons attaché un modèle 3D d'une arme AK47 téléchargé en ligne avec la texture associée.

Dans un premier temps, nous n'avions pas automatiquement attaché l'arme à la caméra lors de l'importation de l'arme. Nous avons commencé par définir la position de l'arme par rapport à celle de la caméra « à la main », c'est à dire, à chaque itération, mettre à jour la position de l'arme en appliquant une transformation (Rotation et Translation) en fonction de la nouvelle position de la caméra et de son orientation. Ce procédé s'est avéré assez complexe car de nombreux cas étaient à traiter en fonction du positionnement du joueur et donc de la caméra.

Par la suite, en s'attardant sur la documentation de la fonction *addAnimatedMeshSceneNode*, nous nous sommes aperçu qu'il était possible de directement fixer notre modèle à la caméra active tout en lui appliquant une translation et rotation relative à celle-ci, une sorte d'offset. Nous avons dorénavant une vue FPS avec une arme qui suit nos déplacements.

Nous avons aussi implémenté différentes méthodes permettant par exemple de gérer les points de vie du joueur (*getHealth*, *setHealth* ou encore *healthDecrease*). Nous avons aussi différents setter et getter pour, entre autres, obtenir le node, savoir si le joueur est en vie ainsi que modifier la position de l'arme etc.

Dans cette classe, nous avons intégré la fonction *targetGun* permettant, à partir d'un rayon partant du joueur, de nous donner, si collision il y a, le node intersecté. Cette méthode nous sera très utile par la suite pour savoir si nous tirons sur un ennemi.

4. Game:

Le but du jeu étant d'éliminer les ennemis, nous avons décidé d'associer le clic gauche de notre souris à un tir. Nous capturons les clics dans notre classe *GameEvent* puis nous vérifions si notre tir a touché un ennemi dans la classe *Game*. Pour cela nous utilisons notre fonction *targetGun* de la classe *Player* qui nous permet, à partir d'un rayon, de nous donner, si collision il y a, le node intersecté. Nous vérifions alors si le node touché correspond à un ennemi. Si tel est le cas, nous diminuons sa vie de celui-ci (*healthDecrease*) ou nous le supprimons s'il n'en a plus en l'effaçant de la liste des ennemis et en supprimant le node.

La perte de points de vie est associée à un changement d'animation et de texture de l'ennemi. En effet, lorsque *healthDecrease* est appelée, l'ennemi passe à la couleur rouge et joue une animation représentant la souffrance pendant un délai de 0.2s. Dans *Game*, nous vérifions si ce délai est écoulé grâce à une condition sur le temps et nous une boucle qui risquerait de bloquer le reste du code.

Nous utilisons le même procédé pour le recul de l'arme. Dans le *main*, après un clic droit (hors menu), nous appliquons une translation de l'arme puis après un délai de 0.2s, nous appliquons la transformation opposée pour simuler le recul en faisant appel à une fonction *setRecul* de la classe *Player*.

A l'aide de la classe *Enemy*, on peut créer une instance avec les caractéristiques définies. Pour la création d'une instance, on a besoin de certains paramètres comme la position. On a défini une liste de position dans la classe *Game* où on sélectionne une, aléatoirement.

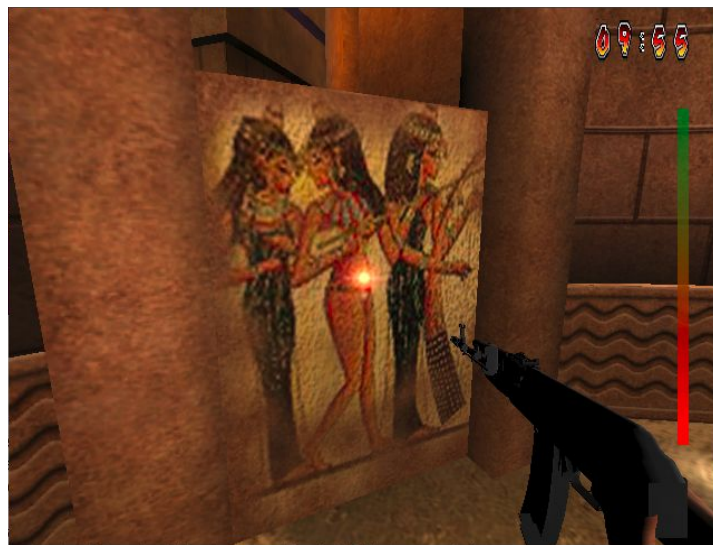
On a alors créé une fonction *loadEnemy()* permettant de créer des ennemies et de les mettre sur une liste qui sera par la suite utilisé pour les afficher dans le jeu. Suivant la vague de zombies, on a un certain nombre d'ennemies. Pour la dernière vague qui sera composé de 12 zombies, on a créé des instances *Enemy* avec un affichage transparent.

Après ces étapes, on a géré les collisions entre zombies et les collisions entre joueur et zombies. On a juste utilisé le bounding box. Il s'agit d'une "boîte" ayant une forme géométrique simple et englobant une forme géométrique. Pour les collisions entre 2 maillages, on effectue le test sur les bounding box qui les entourent en utilisant simplement la fonction *intersectsWithBox()*. Comme leurs formes sont beaucoup plus simples, les calculs sont beaucoup plus rapides. Pour chaque zombie, on a accès à son bounding box,

pour le joueur aussi aussi on a créé un box pour gérer les collisions. La collision entre zombie et joueur va réduire la vie de joueur qui est initialement égale 10.

De ces différentes parties précédentes, on peut ainsi jouer. On a la collision, on peut se déplacer, éliminer les ennemis et subir des dégâts en cas de contacts avec les zombies. On a ajouté l'affichage de la vie et du temps. Pour l'affichage de la vie, on ne fait dessiner un rectangle dans la fonction *displayHealthBar()*. Le rectangle sera rempli par des couleurs allant du vert au rouge en utilisant une interpolation. Pour temps, on possède un ensemble d'images de nombre de 0 à 9. A l'aide de quelques opérations, on peut réaliser un timer de 600 secondes (10 minutes). On utilise le débit d'affichage pendant le jeu en FPS pour décrémenter le temps. Généralement, on a 60 frames par secondes, donc on a créé une variable qui est incrémenté durant le jeu, s'il atteint 60, on a alors une seconde et on peut décrémenter le temps. On l'a généralisé en utilisant directement une fonction donnant le débit d'affichage du jeu. Pour l'affichage du temps, on part du principe que pour les minutes et les secondes, on a l'unité et la dizaine (entre 0 et 9). Il ne faut alors déterminer ces unités et dizaines qui correspondent à nos images et permettent l'affichage. Les minutes correspondent au résultat de la division entière entre le temps en seconde et 60, et plus précisément pour la dizaine, on prend le résultat précédent, on fait sa division entière avec 10 et pour l'unité, on prend le quotient de cette division. On fait de même pour les secondes, par contre on ne prend plus le résultat de la division par 60 mais le quotient.

On a ajouté une fonction pause



On a ajouté une fonction pause, en appuyant sur ECHAP. On a accès aux instructions, aux commandes et on peut rejouer.



Quand le jeu est terminé c'est à dire quand toutes les vagues de zombies on était éliminé, ou que le joueur est "mort", on a ajouté un bouton "recommencer" pour relancer le jeu. Pour cela, on fait dans la classe, une mise à jour des différents paramètres, le temps, la vague, la vie du joueur, puis on écrase la liste des ennemies pour une nouvelle initialisation.

5. Irrklang :

Vous verrez qu'on a ajouté aussi la bibliothèque irrklang pour la gestion du son. Durant le jeu, on a une musique qu'on a choisie. Quand on tire aussi, un son de tir d'arme est déclenché.

III. Perspectives

Nous sommes très satisfaits de notre jeu, pourtant au départ nous avons envisagé certaines features et mécaniques de jeu que nous n'avons malheureusement pas eu le temps d'implémenter.

Nous voulions, par exemple, ajouter des objets à ramasser par le joueur qui auraient pu lui conférer des bonus comme un boost de vie ou de puissance, voire même une nouvelle arme. Pour cela, il faudrait ajouter des nouveaux nodes correspondant aux objets et gérer la collision avec le joueur tout en laissant celui les traverser. Une fois l'objet ramassé, il nous aurait suffi de mettre à jour la barre de vie du joueur dans le cas d'un boost de vie. Dans le cas de la puissance, nous aurions pu modifier le nombre de dégâts infligés aux ennemis.

Nous avons aussi dans l'idée d'avoir un deuxième mode de visée, comme ce qu'on trouve communément dans ce genre de jeu. Pour cela, lors d'un clic droit, nous aurions pu garder la même arme mais la positionner et l'orienter différemment de manière à voire la scène depuis le réticule de l'arme. Ou alors, utiliser un modèle de ridicule de type "scope".

Conclusion

Pour conclure, nous sommes partis d'une idée de jeu qui nous plairait et nous avons réussi à le développer. Tout a commencé avec le choix d'une map 3D, puis du modèle de l'ennemi et du joueur. Nous avons alors pu mettre en place notre dynamique de jeu : des vagues d'ennemis à anéantir. Pour cela, nous avons choisi notre caméra correspondant au joueur et

lui affecter des comportements possibles (déplacements et saut) ainsi que sa relation avec son environnement et les ennemis. Nous avons donc appris à rendre les ennemis dynamiques et leur conférer des caractéristiques comme de la vie et des animations.

Nous avons géré les menus, les tirs du joueur et la musique d'ambiance.

Ainsi ce jeu créé grâce à irrlicht, nous a permis, à partir d'une idée, de mettre en place tout ce que nous désirions pour obtenir une expérience la plus satisfaisante possible.